APACHE HADOOP

FUNDAMENTALS

By Michael Fernandes

TABLE OF CONTENTS

Made by Michael Fernandes

5

YARN Fundamentals

- Introduction to YARN
- Need For Yarn
- YARN Architecture
- YARN Workflow
- Scheduling in YARN

6

MapReduce Fundamentals

- Introduction to MapReduce
- Features of MapReduce
- MapReduce Architecture
- Word Count Example

7

Hadoop Use Cases

8

Hadoop Ecosystem Tools

- Apache Hive
- Apache Pig
- Apache HBase
- Apache Sqoop
- Apache Flume
- Apache Kafka
- Apache Oozie
- Apache ZooKeeper
- Apache Mahout

Part I - Introduction to Big Data

1.1 What is Big Data

Big Data refers to extremely large and complex datasets that cannot be efficiently stored, processed, or analyzed using traditional database systems and computing methods.

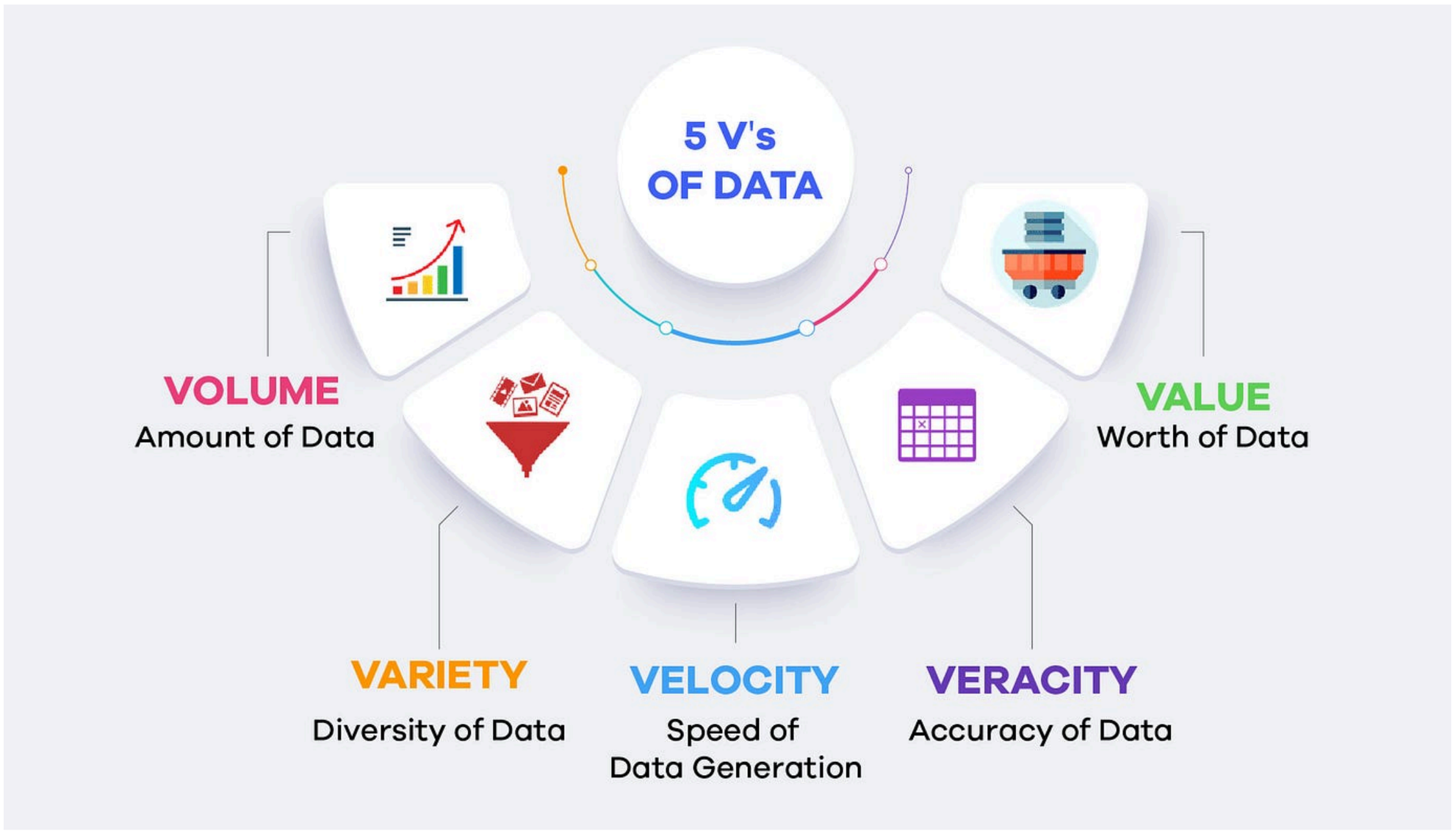
These datasets are generated from various sources such as:

- Social media platforms
- Financial transactions
- IoT devices and sensors
- Web applications
- Machine logs

Big Data requires distributed storage systems and parallel processing frameworks to handle:

- Massive scale
- High speed of data generation
- Different data formats
- Real-time analytics requirements

1.2 Characteristics of Big Data (5 V's)



1. Volume

Refers to the enormous amount of data generated every second.
Examples

- Petabytes of social media data ,
- Financial market transactions
- Sensor-generated data

Challenge - Traditional databases cannot scale efficiently for such huge datasets.

2. Velocity

Refers to the speed at which data is generated, processed, and analyzed.
Examples

- Stock market feeds
- Real-time streaming
- IoT sensor updates

Challenge - Systems must process data quickly to provide real-time insights.

3. Variety

Refers to different types and formats of data.
Types
- Structured data
- Semi-structured data
- Unstructured data

Examples
- Images
- Videos
- JSON files
- Emails
- Audio files

Challenge - Managing multiple formats in one system.

4. Veracity

Refers to the quality, reliability, and accuracy of data.
Problems
- Missing values
- Duplicate records
- Noisy data
- Inconsistent data

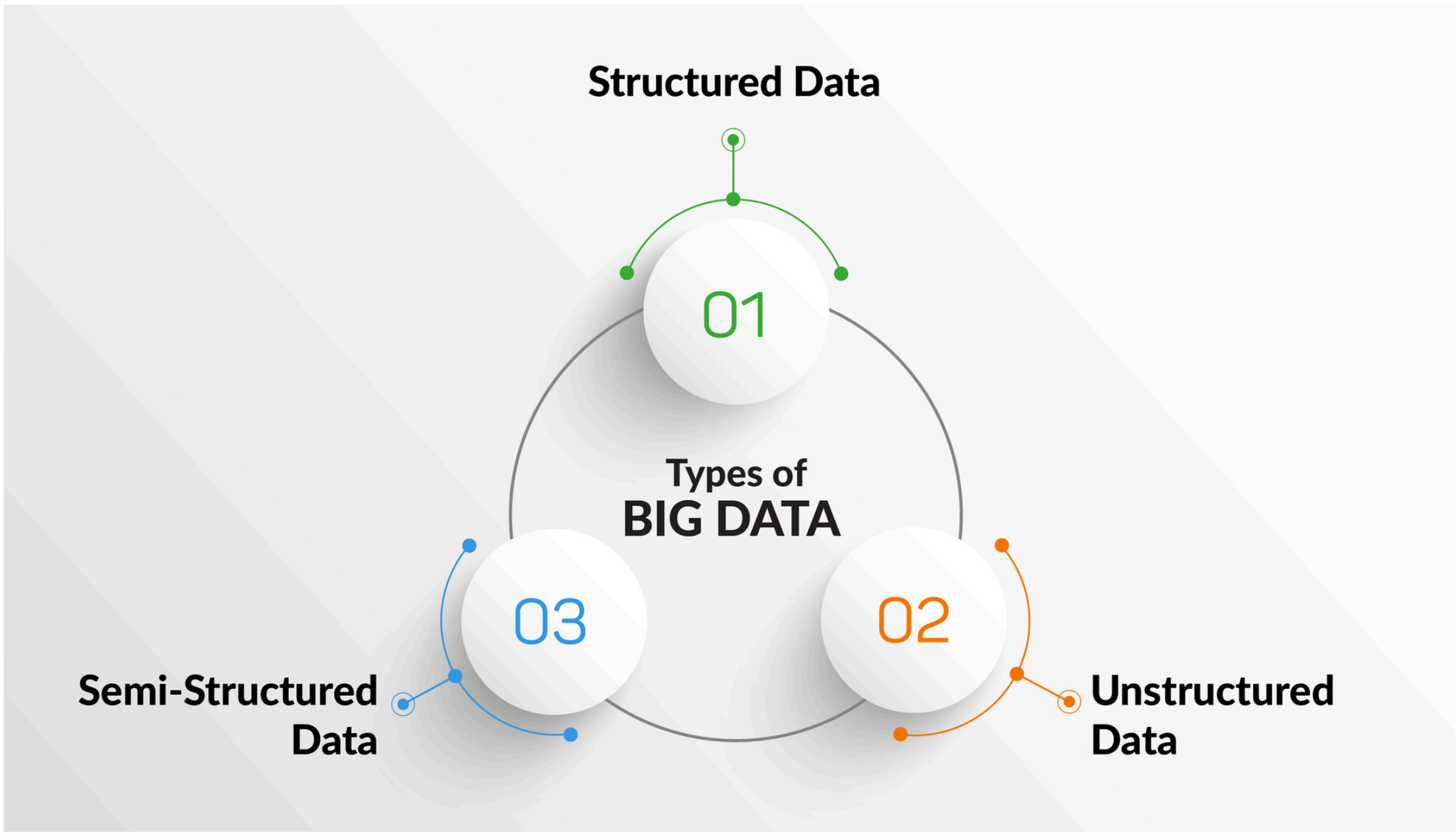
Importance - Poor-quality data leads to incorrect analytics and decisions.

5. Value

Refers to extracting useful insights and business benefits from data.
Examples
- Fraud detection
- Customer recommendations
- Risk analysis
- Predictive analytics

Importance - Data becomes valuable only when meaningful insights are generated.

1.3 Types of Data



1. Structured Data

Data organized in a fixed schema with rows and columns.

Characteristics
- Easily searchable
- Stored in relational databases
- Highly organized

Examples
- Banking records
- Employee databases
- Sales transactions

Technologies
- MySQL
- PostgreSQL
- Oracle

2. Semi-Structured Data

Data that does not follow a strict relational structure but contains tags or markers.

Characteristics

- Flexible schema
- Partially organized

Examples

- JSON
- XML
- Emails

Technologies

- MongoDB
- NoSQL systems

3. Unstructured Data

Data without any predefined structure or schema.

Characteristics

- Difficult to process traditionally
- Largest share of modern data

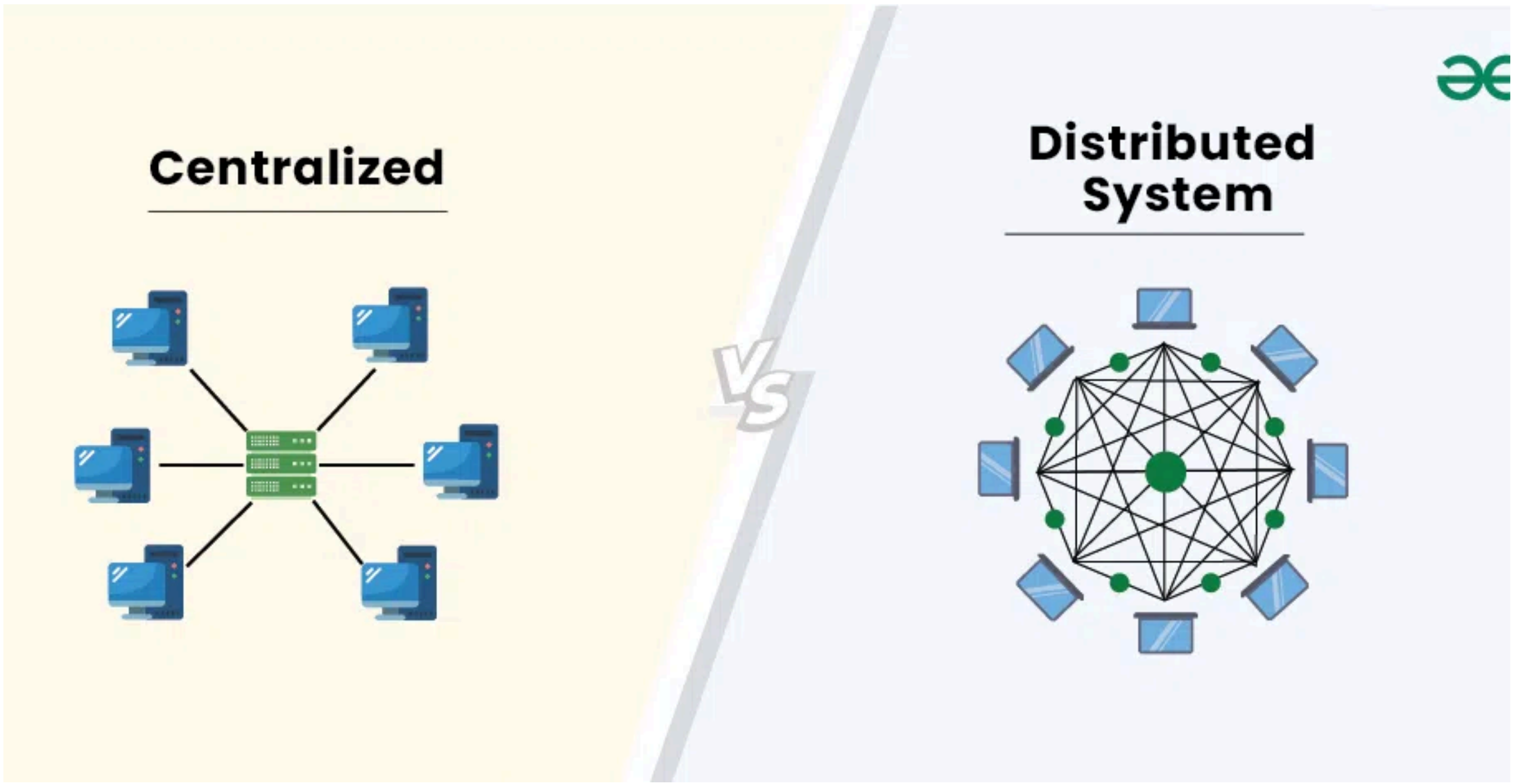
Examples

- Videos
- Images
- Social media posts
- Audio files
- PDFs

Challenge

Requires advanced distributed processing systems.

1.4 Traditional vs Distributed Systems



Traditional System	Distributed System
Single machine architecture	Multiple interconnected machines
Vertical scaling	Horizontal scaling
Limited storage capacity	Massive scalable storage
Expensive hardware upgrades	Commodity hardware usage
Single point of failure	Fault-tolerant architecture
Lower parallelism	High parallel processing
Difficult to handle Big Data	Designed for Big Data workloads

1.5 Need For Hadoop

Hadoop was developed to solve the limitations of traditional systems when dealing with Big Data.

1. Massive Data Storage
Traditional databases struggle with petabyte-scale datasets.
HDFS stores data across multiple machines in a distributed manner.

2. Scalability
Scaling traditional systems requires expensive hardware upgrades.
Hadoop supports horizontal scaling by adding more nodes to the cluster.

3. Fault Tolerance
Single-server failures can cause system downtime.
Data replication ensures data availability even if nodes fail.

4. Parallel Processing
Traditional systems process data sequentially.
MapReduce enables distributed parallel processing across multiple nodes.

5. Cost Efficiency
High-end enterprise servers are expensive.
Uses low-cost commodity hardware.

6. Handling Variety of Data
Traditional databases mainly handle structured data.
Hadoop Can process Structured, Semi Structured and Unstructured Data.

7. High Throughput Processing
Big Data workloads require fast data access and processing.
Designed for high-throughput distributed processing rather than low-latency transactions.

Part II - Introduction to Apache Hadoop

2.1 What is Hadoop

Apache Hadoop is an open-source framework designed for distributed storage and distributed processing of massive datasets across clusters of commodity hardware.

It allows organizations to:

- Store huge amounts of data
- Process data in parallel
- Scale systems easily
- Handle hardware failures efficiently

It is primarily used for:

- Big Data analytics
- Batch data processing
- Data warehousing
- Log analysis
- Machine learning pipelines

Instead of using one powerful machine:

- Hadoop distributes data across multiple machines
- Processing is performed in parallel near the data location

This provides:

- Faster processing
- Scalability
- Fault tolerance
- Cost efficiency

2.2 Features of Hadoop

1. Distributed Storage
Hadoop stores data across multiple nodes using HDFS (Hadoop Distributed File System).
Benefits
- Large-scale storage
- Data redundancy
- Fault tolerance

2. Distributed Processing
Hadoop processes data in parallel using MapReduce and YARN.
Benefits
- Faster execution
- Efficient resource utilization
- Parallel computation

3. Scalability
Hadoop can scale horizontally by adding more machines to the cluster.
Types
- Small clusters
- Thousands of nodes

4. Fault Tolerance
Data is replicated across multiple DataNodes.
Mechanism
- Automatic replication
- Node failure recovery
- Task re-execution

5. High Throughput
Hadoop is optimized for processing large datasets efficiently.
Suitable For
- Batch processing
- Analytics workloads

6. Cost Efficiency
Uses low-cost commodity hardware instead of expensive enterprise servers.

7. Flexibility
Supports:

- Structured data
- Semi-structured data
- Unstructured data

2.3 Advantages of Hadoop

1. Massive Scalability
Can scale from a few nodes to thousands of nodes.

2. Fault-Tolerant Architecture
Data replication ensures availability even during hardware failures.

3. Parallel Processing
Multiple nodes process data simultaneously.

4. Cost Effective
Runs on commodity hardware

5. Flexible Data Handling
Supports multiple data formats:

- Text
- Images
- Videos
- Logs
- JSON
- XML

6. Open Source
- Free to use
- Large community support
- Extensive ecosystem

7. High Availability
Distributed architecture minimizes downtime.

8. Suitable for Big Data Analytics
Efficient for:
- ETL pipelines
- Data lakes
- Batch analytics

2.4 Limitations of Hadoop

1. High Latency
Not suitable for real-time processing as MapReduce is disk-based and batch-oriented.

2. Poor Small File Handling
Large numbers of small files overload the NameNode metadata memory.

3. Complex Development
Writing MapReduce programs can be complicated.

4. Not Suitable for OLTP
Hadoop is designed for analytical workloads, not transactional systems.

5. Limited Real-Time Processing
Better suited for batch processing than streaming analytics.

6. Security Complexity
Configuring Hadoop security (Kerberos, ACLs, encryption) is complex.

7. NameNode Bottleneck
The NameNode stores metadata centrally, creating a potential bottleneck

8. Resource Intensive

It Requires:

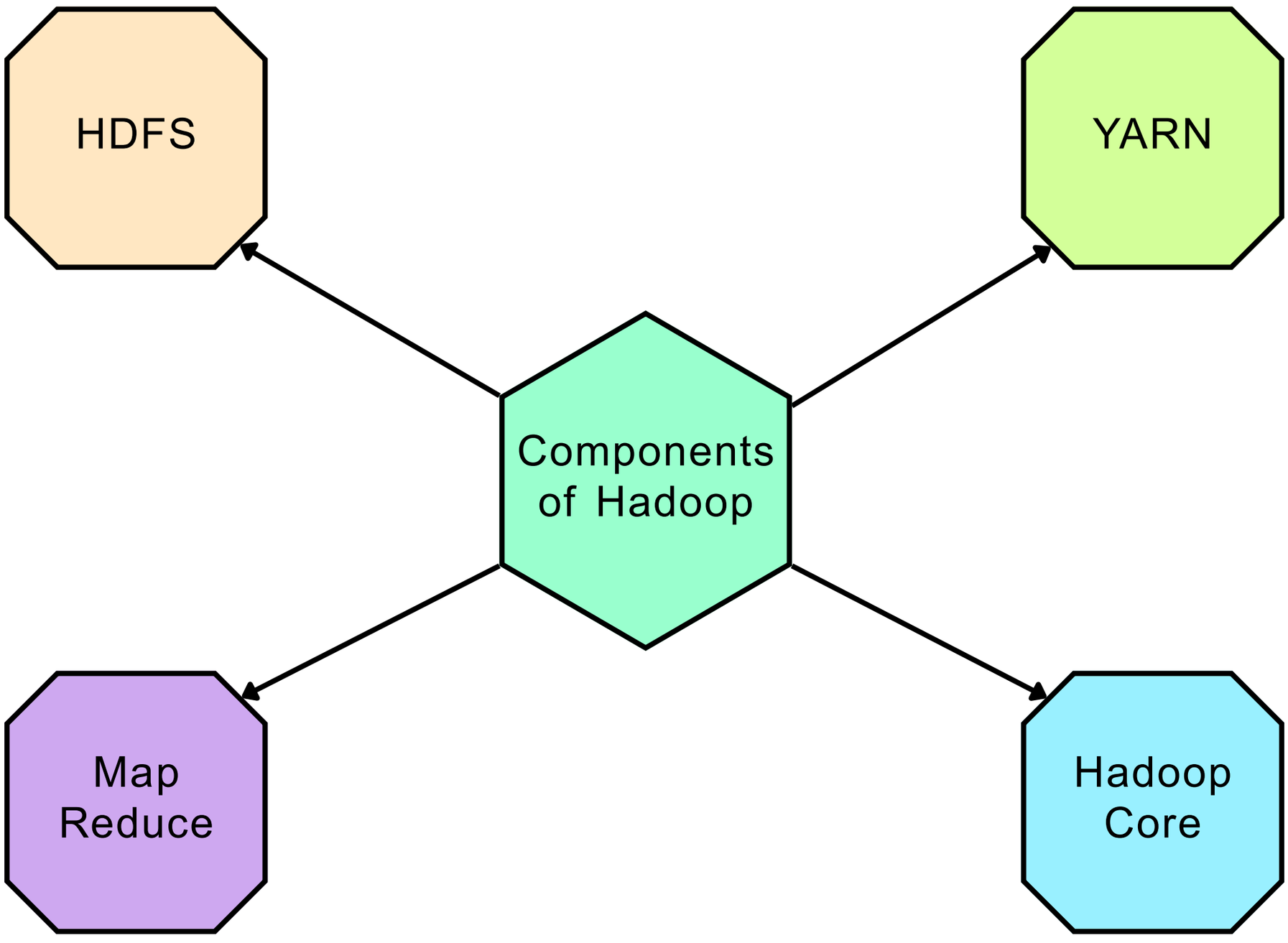
- Large storage
- Multiple machines
- Cluster maintenance

8. Suitable for Big Data Analytics

Efficient for:

- ETL pipelines
- Data lakes
- Batch analytics

2.5 Components of Hadoop



1. HDFS (Hadoop Distributed File System)

HDFS is Distributed storage system for large datasets.

Functions
Stores files across multiple nodes
Splits files into blocks
Replicates data for fault tolerance

Main Components
NameNode
DataNode

2. YARN (Yet Another Resource Negotiator)

YARN is used for Cluster resource management and job scheduling.

Functions
Allocates system resources
Manages containers
Schedules tasks

Main Components
ResourceManager
NodeManager
ApplicationMaster

3. MapReduce

MapReduce is used for Distributed parallel data processing framework.

Functions
Processes data in parallel
Divides tasks into Map and Reduce phases

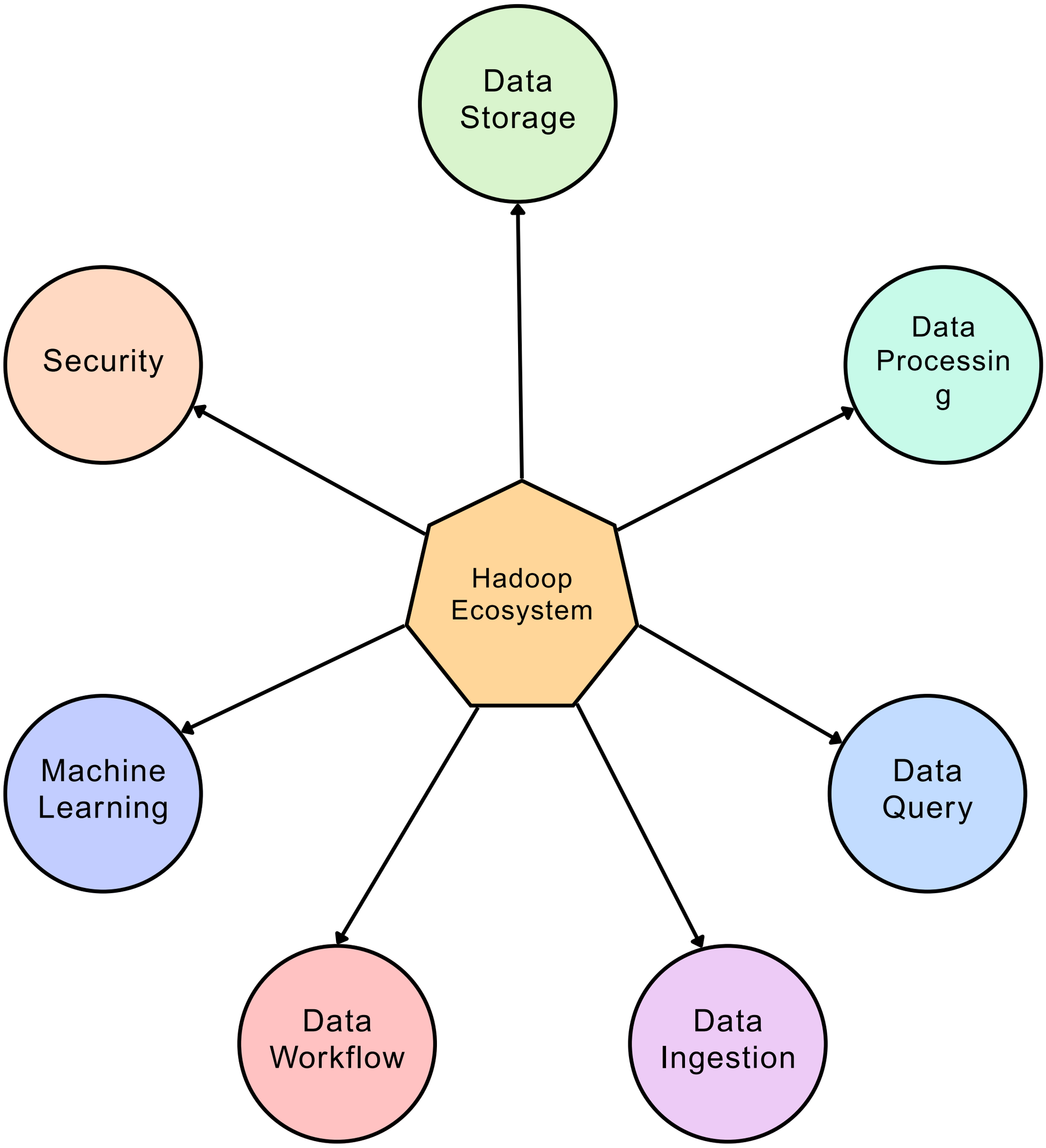
Phases
Mapping
Shuffle and Sort
Reducing

4. Hadoop Core

It provides shared libraries and utilities required by Hadoop modules.

Includes
Java libraries
Scripts
APIs
Configuration files

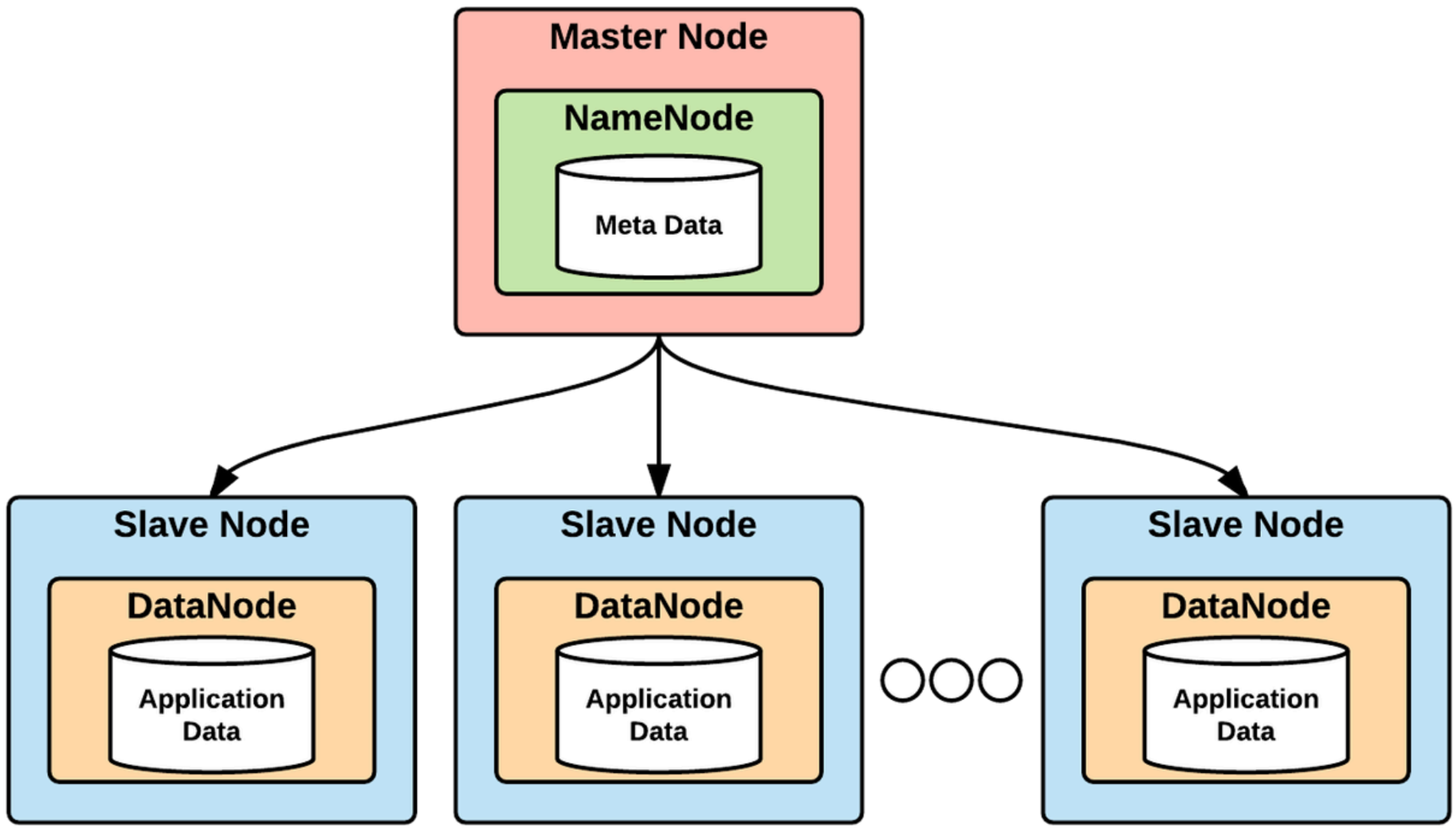
2.5 Hadoop Ecosystem



Data Storage	HDFS Distributed storage layer.
	HBase NoSQL column-family database built on Hadoop.
Data Processing	MapReduce Batch processing framework.
	Apache Spark Fast in-memory processing engine.
	Apache Tez Optimized DAG-based processing engine.
Data Query	Apache Hive SQL-like data warehouse system
	Apache Pig Data flow scripting platform.
Data Ingestion	Apache Sqoop Transfers data between Relational databases and Hadoop systems
	Apache Kafka Distributed event streaming platform.
	Apache Flume Collects and transfers log data into Hadoop.

Data Workflow	Apache Oozie Workflow scheduler for Hadoop jobs.
	Apache ZooKeeper Distributed coordination service. Functions • Synchronization • Configuration management • Leader election
Machine Learning	Apache Mahout Machine learning library for scalable analytics.
Data Security	Apache Ranger Centralized security administration.
	Kerberos Authentication mechanism for Hadoop security.

Part III - Hadoop Architecture



3.1 Master-Slave Architecture

Master-slave architecture is a distributed computing model in which one or more master nodes manage and coordinate multiple slave (worker) nodes.

In Hadoop:
Masters manage metadata, resources, and scheduling
Slaves store data and execute tasks

Master Nodes

Responsible for:
- Cluster management
- Metadata management
- Resource allocation
- Job scheduling

Main Master Components
- NameNode
- ResourceManager

Slave (Worker) Nodes

Responsible for:
- Data storage
- Task execution

Main Worker Components
- DataNode
- NodeManager

Advantages of Master-Slave Architecture

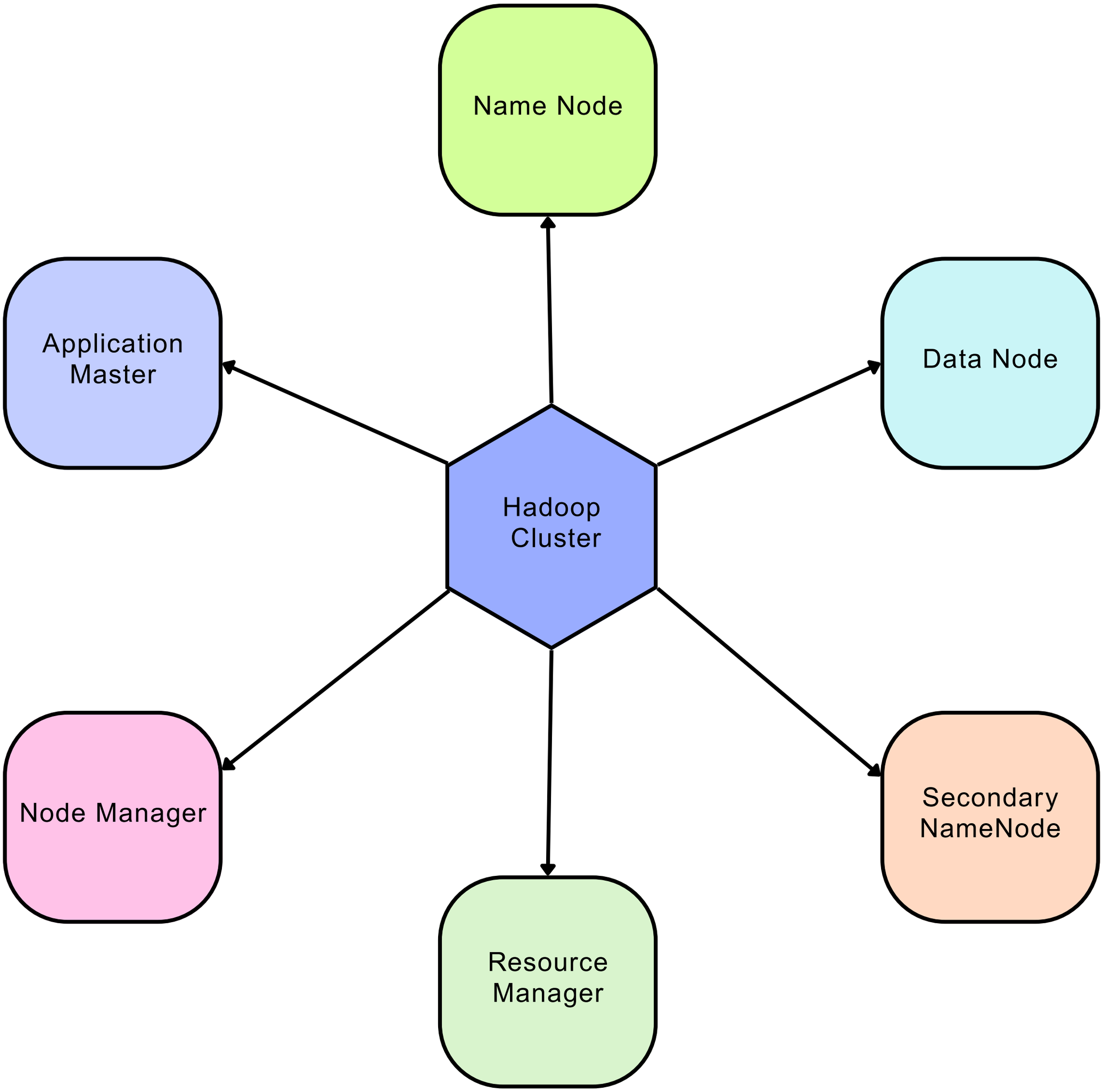
1. Scalability
New worker nodes can be added easily.

2. Parallel Processing
Tasks are distributed across multiple nodes.

3. Fault Tolerance
Failure of one worker node does not stop the cluster.

4. Efficient Resource Management
Masters coordinate workloads efficiently.

3.2 Hadoop Cluster Components



Name Node

The NameNode is the master server of HDFS responsible for managing file system metadata.

- Maintains file system namespace
- Stores metadata
- Tracks block locations
- Manages file permissions
- Handles client requests

Metadata Stored by NameNode

- File names
- Directory structure
- Block locations
- Replication information
- Access permissions

Data Node

DataNodes are worker nodes responsible for storing actual data blocks.

- Store HDFS blocks
- Perform read/write operations
- Send heartbeats
- Send block reports
- Replicate data blocks

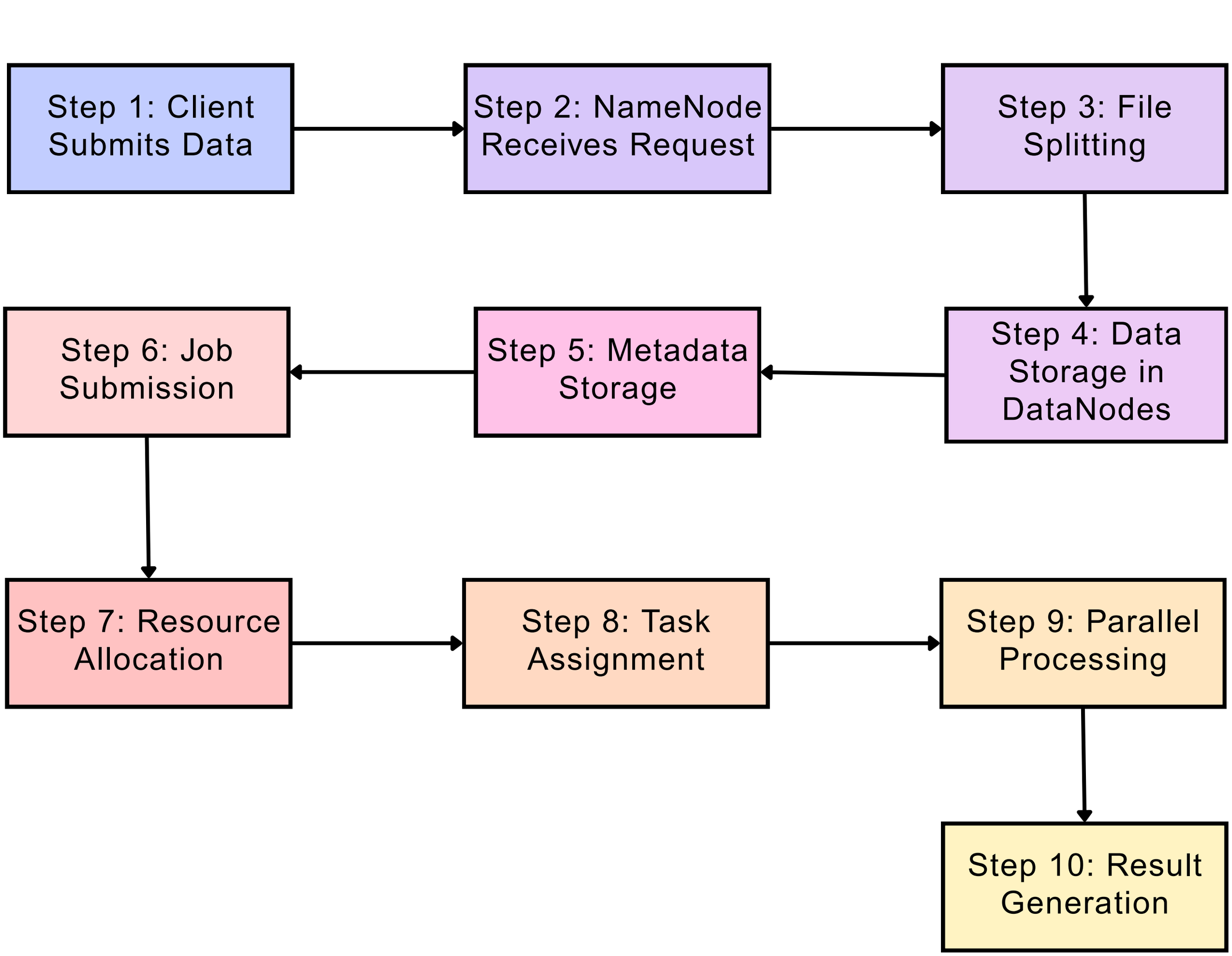
Characteristics

- Multiple DataNodes exist in a cluster
- Each stores portions of files

Secondary Name Node	The Secondary NameNode performs checkpointing operations. Responsibilities • Merges FsImage and EditLogs • Reduces NameNode metadata size • Assists NameNode recovery
Resource Manager	The ResourceManager is the master component of YARN. Responsibilities • Resource allocation • Job scheduling • Cluster resource monitoring Main Components Scheduler Allocates cluster resources. Application Manager Manages application execution.
Node Manager	NodeManager runs on worker nodes and manages node-level resources. Responsibilities • Container management • Resource monitoring • Task execution • Reporting node status

Application Master	ApplicationMaster manages a specific application/job in YARN. Responsibilities • Negotiates resources • Coordinates tasks • Monitors execution

3.3 Hadoop Cluster Workflow



Step 1: Client Submits Data
The client uploads a file to HDFS.

Step 2: NameNode Receives Request
The NameNode:

- Checks permissions
- Creates metadata entries
- Determines DataNode block placement

Step 3: File Splitting
The file is divided into blocks.
Example
A 1 GB file may be split into:

- 128 MB blocks

Step 4: Data Storage in DataNodes
Blocks are distributed across multiple DataNodes.
Replication
Each block is replicated based on the replication factor.
Example:

- Replication factor = 3
- Three copies stored on different nodes

Step 5: Metadata Storage
The NameNode stores:

- File name
- Block IDs
- DataNode locations

Step 6: Job Submission
The client submits a processing job to YARN.

Step 7: Resource Allocation
The ResourceManager:

- Allocates resources
- Launches ApplicationMaster

Step 9: Parallel Processing
Tasks execute on nodes where data exists.
This is called Data Locality

Step 10: Result Generation
Processed output is written back to HDFS.

3.4 Hadoop Cluster Workflow

Heartbeats

A heartbeat is a periodic signal sent by DataNodes to the NameNode.

It Indicates that the DataNode is alive and functioning properly.
Information Included
- Node health status
- Available storage
- Processing status

Failure Detection
If the NameNode does not receive heartbeats within a specified time:
- The DataNode is marked dead
- Replication of lost blocks begins

Block Reports

Block reports are detailed reports sent by DataNodes to the NameNode.

It Provide information about all blocks stored on a DataNode.
Information Included
- Block IDs
- Block locations
- Replica information

Importance
Helps the NameNode:
- Track data blocks
- Verify replication
- Detect missing blocks

Part IV - HDFS Fundamentals

4.1 Introduction to HDFS

HDFS (Hadoop Distributed File System) is the distributed storage layer of Hadoop designed to store massive datasets across multiple machines in a cluster.

It is optimized for:

- High-throughput data access
- Fault tolerance
- Distributed storage
- Large-scale data processing

HDFS follows a Write Once, Read Many (WORM) model, meaning files are generally written once and read multiple times.

Key Characteristics of HDFS

- Distributed file system
- Stores data across multiple nodes
- Handles hardware failures automatically
- Designed for very large files
- Supports parallel access
- Uses commodity hardware

4.2 Design Goals of HDFS

1. Fault Tolerance
Hardware failures are common in large clusters.
HDFS Solution
   - Data replication
   - Automatic recovery
   - Failure detection using heartbeats

2. High Throughput Access
HDFS is optimized for:
- Large sequential reads
- Batch processing

Suitable For
- Analytics
- ETL workloads
- MapReduce jobs

3. Scalability
HDFS can scale horizontally by adding more DataNodes.

4. Large File Storage
HDFS is designed for:
- Gigabyte files
- Terabyte files
- Petabyte datasets

5. Data Locality
Processing is moved close to where data resides.
Benefit
Reduces network traffic and improves performance.

6. Cost Efficiency
Uses low-cost commodity hardware instead of expensive storage systems.

7. Streaming Data Access
Optimized for continuous streaming access rather than random access.

4.3 HDFS Blocks

HDFS divides files into fixed-size blocks for distributed storage.

Default Block Size
Common values:

- 128 MB
- 256 MB

Example
A 500 MB file with 128 MB block size becomes:

- Block 1 → 128 MB
- Block 2 → 128 MB
- Block 3 → 128 MB
- Block 4 → 116 MB

4.4 Replication

HDFS stores multiple copies of each block across different DataNodes.
Replication Factor defines the number of block copies.
Default Value of Replication Factor = 3

Example
If Block A has replication factor 3:

- Copy 1 → DataNode 1
- Copy 2 → DataNode 2
- Copy 3 → DataNode 3

4.5 Rack Awareness

Rack awareness is the process of placing replicas across different racks in a cluster.

Why Rack Awareness is Important
If all replicas are stored in one rack:

- Rack failure can cause data loss.

Replica Placement Strategy
Typical strategy:

- One replica in local rack
- Another replica in different rack
- Third replica in same remote rack

4.6 Data Locality

Data locality means moving computation close to the data instead of moving data across the network.

Benefits
1. Reduced Network Traffic
2. Faster Processing
3. Improved Cluster Efficiency

Types of Data Locality
1. Node Locality
Processing occurs on the same node.
2. Rack Locality
Processing occurs within the same rack.
3. Off-Rack Locality
Processing occurs on another rack.

4.7 HDFS Read And Write Operations

HDFS Read Operation

Step 1
Client sends read request to NameNode.
Step 2
NameNode returns block locations.
Step 3
Client connects to nearest DataNode.
Step 4
Data blocks are streamed to client.
Step 5
Client reconstructs the file from blocks.

HDFS Write Operation

Step 1
Client sends file write request to NameNode.

Step 2
NameNode checks permissions and allocates blocks.

Step 3
NameNode selects DataNodes for block storage.

Step 4
Client sends block data to first DataNode.

Step 5
Pipeline replication occurs:

- DataNode 1 → DataNode 2
- DataNode 2 → DataNode 3

Step 6
Acknowledgments are returned after successful storage.

Step 7
Metadata is updated in NameNode.

4.8 HDFS Commands

HDFS commands are used to interact with the Hadoop Distributed File System from the command line.

```
General syntax:hdfs dfs <command>
```

1. File and Directory Commands	List Files and Directories-Displays files and directories in HDFS. Syntax - hdfs dfs -ls <path> Example - hdfs dfs -ls /
	Create Directory Syntax - hdfs dfs -mkdir <directory_name> Example - hdfs dfs -mkdir /data Create Nested Directories `hdfs dfs -mkdir -p /project/input/data`

	Remove Directory
	`Syntax - hdfs dfs -rmdir <directory>`
	`Example - hdfs dfs -rmdir /data`

	Remove Files Syntax - hdfs dfs -rm <file> Example - hdfs dfs -rm /data/file.txt
2. File Upload and Download Commands	Upload File to HDFS Syntax - hdfs dfs -put <local_file> <hdfs_path> Example - hdfs dfs -put data.csv /input
	Download File from HDFS Syntax - hdfs dfs -get <hdfs_file> <local_path> Example - hdfs dfs -get /data/file.txt /home/user

3. File Viewing Commands	Display File Content Syntax - hdfs dfs -cat <file> Example - hdfs dfs -cat /data/file.txt
	View First Few Lines Syntax - hdfs dfs -head <file> Example - hdfs dfs -head /data/file.txt
	View Last Few Lines Syntax - hdfs dfs -tail <file> Example - hdfs dfs -tail /data/file.txt
	Count Words, Lines, Bytes Syntax - hdfs dfs -count <path> Example - hdfs dfs -count /data
4. File Management Commands	Copy Files in HDFS Syntax - hdfs dfs -cp <source> <destination> Example - hdfs dfs -cp /data/file.txt /backup
	Move Files in HDFS Syntax - hdfs dfs -mv <source> <destination> Example - hdfs dfs -mv /data/file.txt /archive
	Rename File Syntax - hdfs dfs -mv oldname.txt newname.txt
	Merge Files Syntax - hdfs dfs -getmerge <source_dir> <local_file> Example - hdfs dfs -getmerge /output result.txt

5. Permission Commands	Change Permissions Syntax - hdfs dfs -chmod <permissions> <file> Example - hdfs dfs -chmod 755 file.txt
	Change Owner Syntax - hdfs dfs -chown <owner> <file> Example - hdfs dfs -chown hadoop file.txt
	Change Group Syntax - hdfs dfs -chgrp <group> <file> Example - hdfs dfs -chgrp admin file.txt
6. Administrative Commands	Check HDFS Report Syntax - hdfs dfsadmin -report
	Enter Safe Mode - hdfs dfsadmin -safemode enter Leave Safe Mode - hdfs dfsadmin -safemode leave
	File System Check Syntax - hdfs fsck / Purpose -Checks HDFS health and block consistency.
7. Replication and Snapshot Commands	Change Replication Factor Syntax - hdfs dfs -setrep <factor> <file> Example - hdfs dfs-setrep 3 /data/file.txt
	Create Snapshot Syntax - hdfs dfs -createSnapshot <directory>
	Delete Snapshot Syntax - hdfs dfs -deleteSnapshot <directory> <snapshot_name>

Part V - YARN Fundamentals

5.1 Introduction to YARN

YARN (Yet Another Resource Negotiator) is the resource management and job scheduling layer of Hadoop introduced in Hadoop 2.x.

It separates:

- Resource management
- Job processing

This allows multiple processing frameworks to run on the same Hadoop cluster.

Definition of YARN

YARN is responsible for:

- Managing cluster resources
- Scheduling applications
- Monitoring execution
- Allocating system resources efficiently

Main Functions of YARN

1. Resource Management
Allocates CPU and memory resources.
2. Job Scheduling
Schedules applications across cluster nodes.
3. Task Monitoring
Monitors application execution.
4. Fault Handling
Detects and recovers failed tasks.
5. Multi-Framework Support
Supports:

- MapReduce
- Spark
- Tez
- Flink
- Storm

5.2 Need For YARN

1. Better Scalability
Supports very large clusters

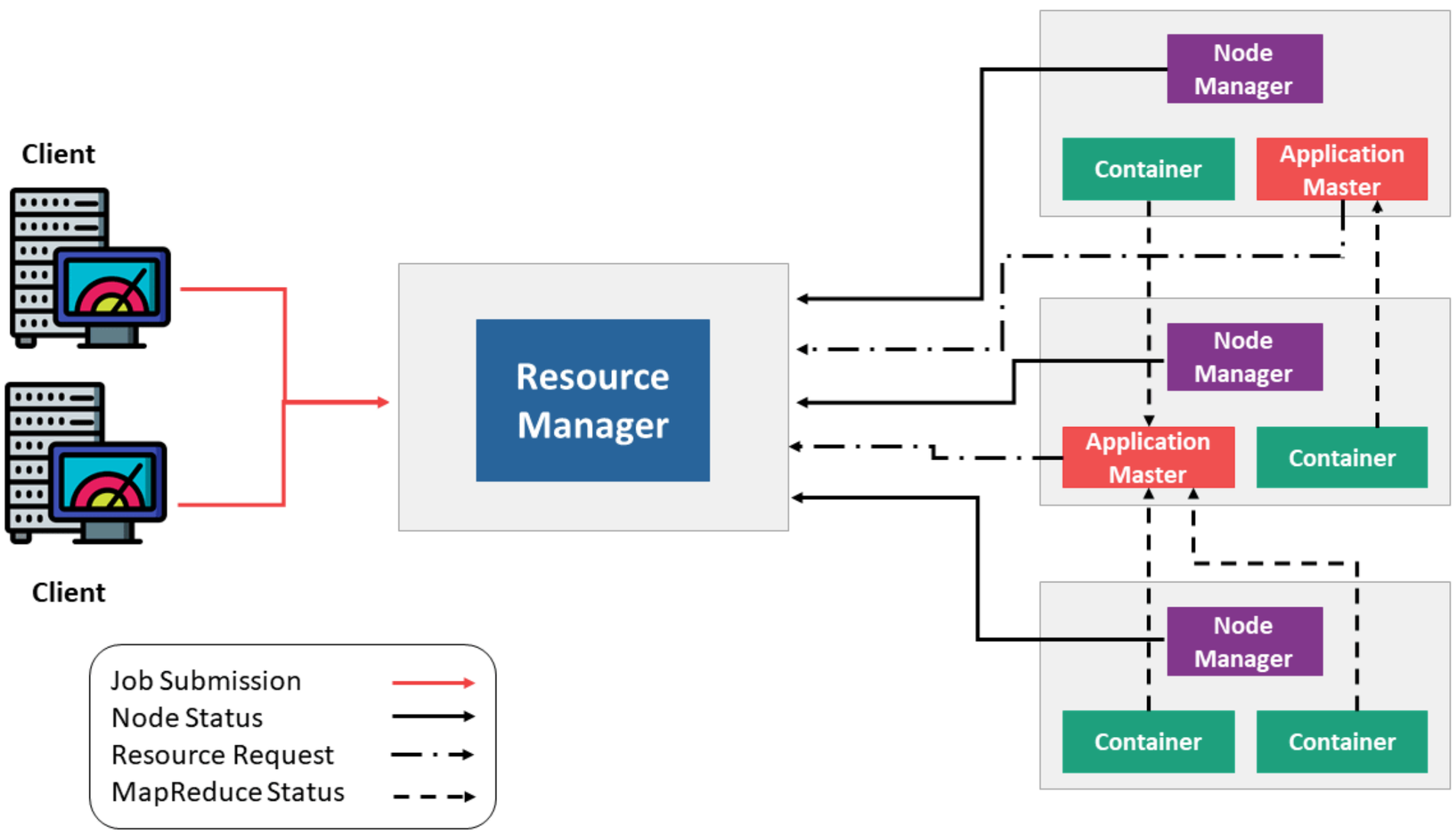
2. Improved Resource Utilization
Dynamic allocation of cluster resources.

3. Multi-Framework Support
Supports diverse workloads.

4. Better Fault Tolerance
Improved recovery mechanisms.

5. Increased Cluster Efficiency
Resources are shared efficiently among applications.

5.3 YARN Architecture



Components of YARN

1. ResourceManager (RM)

Central master service responsible for cluster-wide resource management.

Responsibilities

- Resource allocation
- Job scheduling
- Monitoring cluster resources

Components of ResourceManager

A. Scheduler

Function - Allocates resources to applications.

Characteristics

- No task monitoring
- Pure resource allocation

B. Applications Manager

- Accepts job submissions
- Launches ApplicationMaster
- Restarts failed ApplicationMasters

2. NodeManager (NM)

Runs on each worker node and manages node-level resources.

Responsibilities

- Container management
- Resource monitoring
- Task execution
- Reporting status to ResourceManager

Resources Managed

- CPU
- Memory
- Disk usage

3. ApplicationMaster (AM)

Application-specific manager responsible for executing one application.
Responsibilities

- Negotiates resources
- Coordinates tasks
- Monitors execution
- Handles task failures

4. Containers

Containers are execution environments allocated by YARN.
Includes

- CPU resources
- Memory resources
- Execution environment

It is Used to run tasks securely and independently.

5.4 YARN Workflow

Step 1: Client Submits Application
The client submits a job to the ResourceManager.

Step 2: ResourceManager Allocates Container
ResourceManager allocates a container for launching ApplicationMaster.

Step 3: ApplicationMaster Starts
ApplicationMaster is launched on a NodeManager.

Step 4: Resource Negotiation
ApplicationMaster requests additional resources from ResourceManager

Step 5: Containers Allocated
ResourceManager allocates containers on worker nodes.

Step 6: Task Execution
NodeManagers launch tasks inside allocated containers.

Step 7: Monitoring
ApplicationMaster monitors task progress and handles failures.

Step 8: Completion
Results are stored in HDFS after job completion.

5.5 YARN Scheduling

YARN uses schedulers to allocate cluster resources among applications.
Goals:

- Efficient resource utilization
- Fair sharing
- Multi-user support
- High throughput

Types Of YARN Schedulers

1. FIFO Scheduler

First In First Out scheduling.
The Jobs execute in submission order.
Advantages

- Simple
- Easy to configure

Limitations

- No fairness
- Large jobs may block smaller jobs

2. Capacity Scheduler

Resources are divided into multiple queues.
Features

- Multi-tenant support
- Queue-based resource allocation
- Guaranteed minimum capacity

Suitable For Large organizations with multiple teams.

3. Fair Scheduler

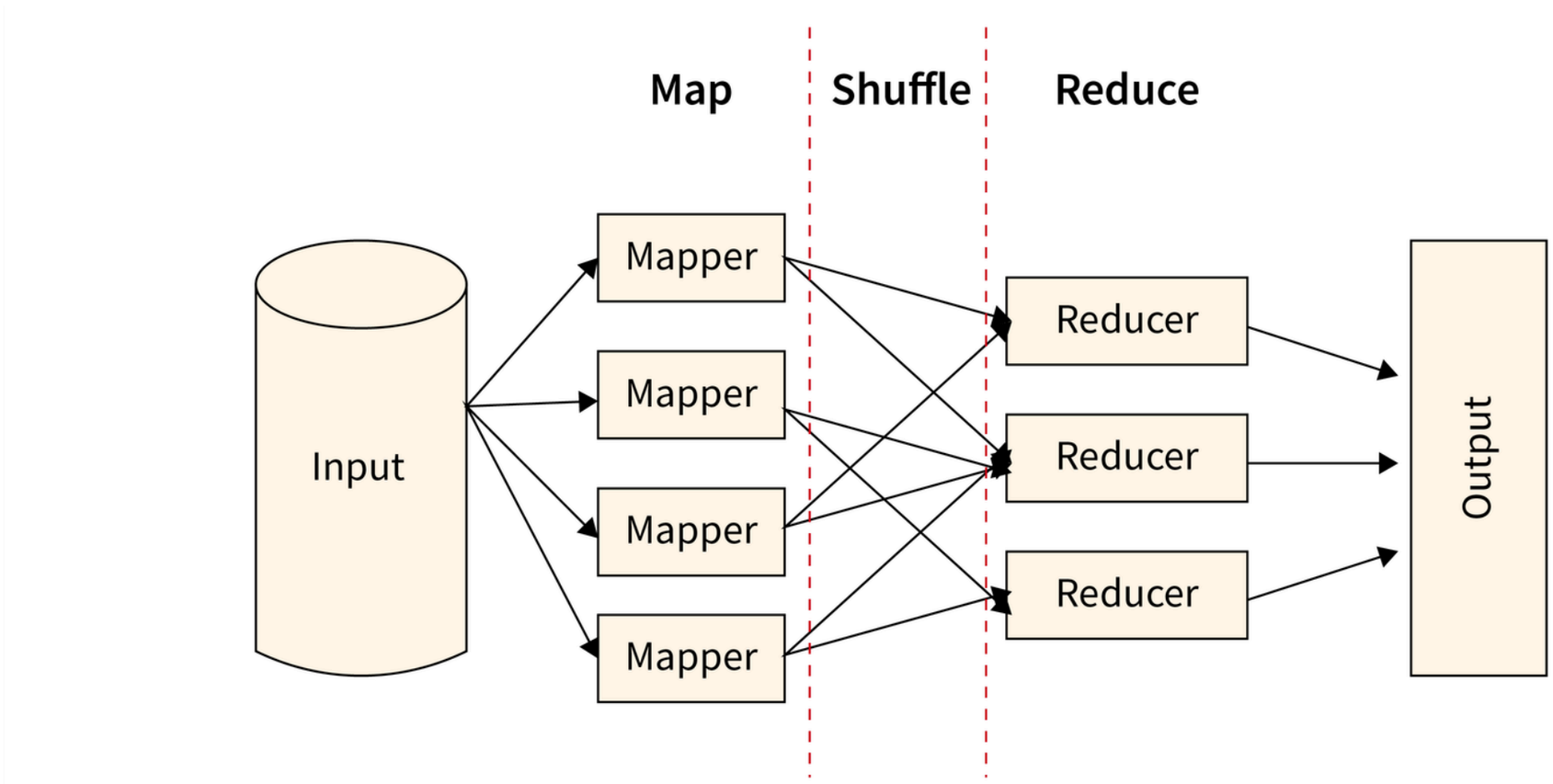
Allocates resources fairly among running applications.
All applications eventually get equal share of resources.
Features

- Dynamic sharing
- Prevents starvation

Suitable For Clusters with many concurrent users.

Part VI - MapReduce Fundamentals

6.1 Introduction to MapReduce



MapReduce is a distributed programming model and processing framework used in Hadoop for processing large datasets across multiple nodes in parallel.

MapReduce processes data in two major phases:

- Map Phase
- Reduce Phase

Purpose of MapReduce
MapReduce is designed for:

- Parallel processing
- Distributed computation
- Large-scale batch processing
- Fault-tolerant data processing

Large datasets are:

1. Divided into smaller chunks
2. Processed in parallel
3. Combined to generate final output

6.2 Features of MapReduce

1. Distributed Processing
Tasks are distributed across multiple cluster nodes.

2. Parallel Execution
Multiple tasks execute simultaneously.

3. Fault Tolerance
Failed tasks are automatically re-executed.

4. Scalability
Can scale to thousands of machines.

5. Data Locality
Processing occurs near data location.

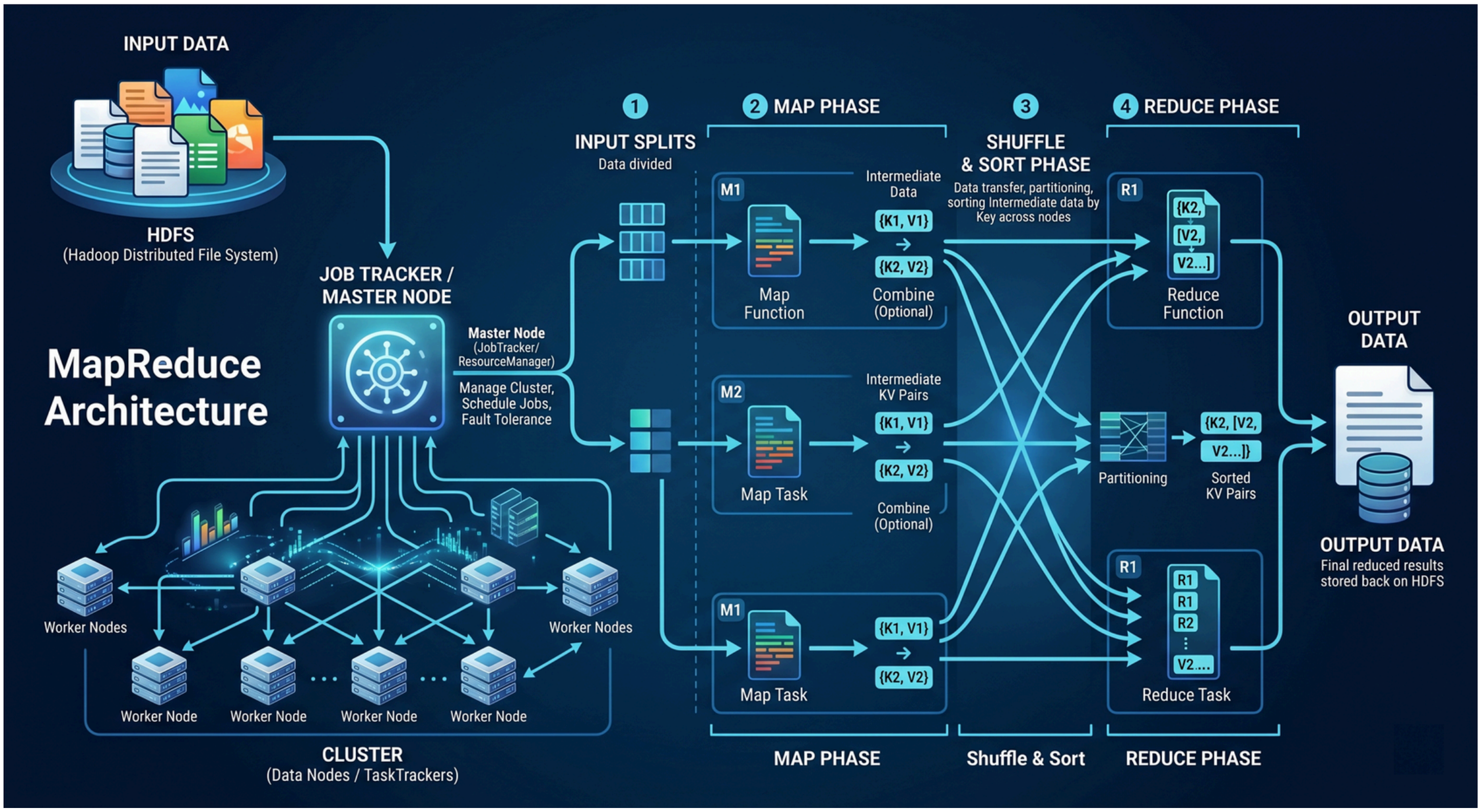
6. Automatic Load Balancing
Tasks are distributed efficiently across nodes.

7. High Throughput
Optimized for large-scale batch analytics.

8. Simple Programming Model
Uses:

- Map function
- Reduce function

6.3 MapReduce Architecture



1. Input Data Phase

Input Files
The process starts with input files stored in HDFS.
Examples:

- Text files
- Logs
- CSV files
- JSON data

File Splitting
Large files are divided into smaller chunks called Input Splits
Example: 1 GB File → 8 Splits of 128 MB

Each split is processed independently.

- Enables parallel processing
- Improves scalability
- Multiple mappers can work simultaneously

2. JobTracker / Master Node

The JobTracker is the master component responsible for managing MapReduce jobs.
Responsibilities of JobTracker
1. Job Scheduling - Assigns tasks to worker nodes.
2. Resource Management - Tracks available cluster resources.
3. Monitoring - Monitors task execution status.
4. Fault Tolerance - Restarts failed tasks.

TaskTracker
Each worker node contains TaskTracker
Responsibilities:

- Executes map and reduce tasks
- Sends heartbeat to JobTracker
- Reports execution status

3. Map Phase

The Map Phase performs initial data processing.
Map Tasks Each Input Split is assigned to one Mapper
Example:

Split 1 → Mapper 1
Split 2 → Mapper 2
Split 3 → Mapper 3

Function of Mapper
The mapper:

- Reads records
- Processes data
- Produces intermediate key-value pairs

Example
Input: Hello Hadoop
Mapper Output:
(Hello,1)
(Hadoop,1)

Parallel Execution
All mappers run simultaneously across multiple nodes.
This provides:

- Faster execution
- Distributed computation

Intermediate Data
Mapper outputs are temporary intermediate key-value pairs.
Example:

(K1,V1)
(K2,V2)

Combiner
A Combiner performs local aggregation before data transfer.
Example
Before Combiner:

(Hello,1)
(Hello,1)
(Hello,1)

After Combiner:

(Hello,3)

4.Shuffle Phase

Intermediate data from all mappers is transferred to reducers.
This involves:

- Network communication
- Data movement across nodes

Partitioning
The partitioner decides which reducer receives which key
Example - Hash(Key) % Number of Reducers

Sorting
Data is grouped by identical keys.
Example
Before Sorting:

(Hello,1)
(Hadoop,1)
(Hello,1)

After Sorting:

(Hello,[1,1])
(Hadoop,[1])

5. Reduce Phase

Reducers process grouped key-value pairs.
Function of Reducer
Reducer:

- Aggregates data
- Performs calculations
- Generates final output

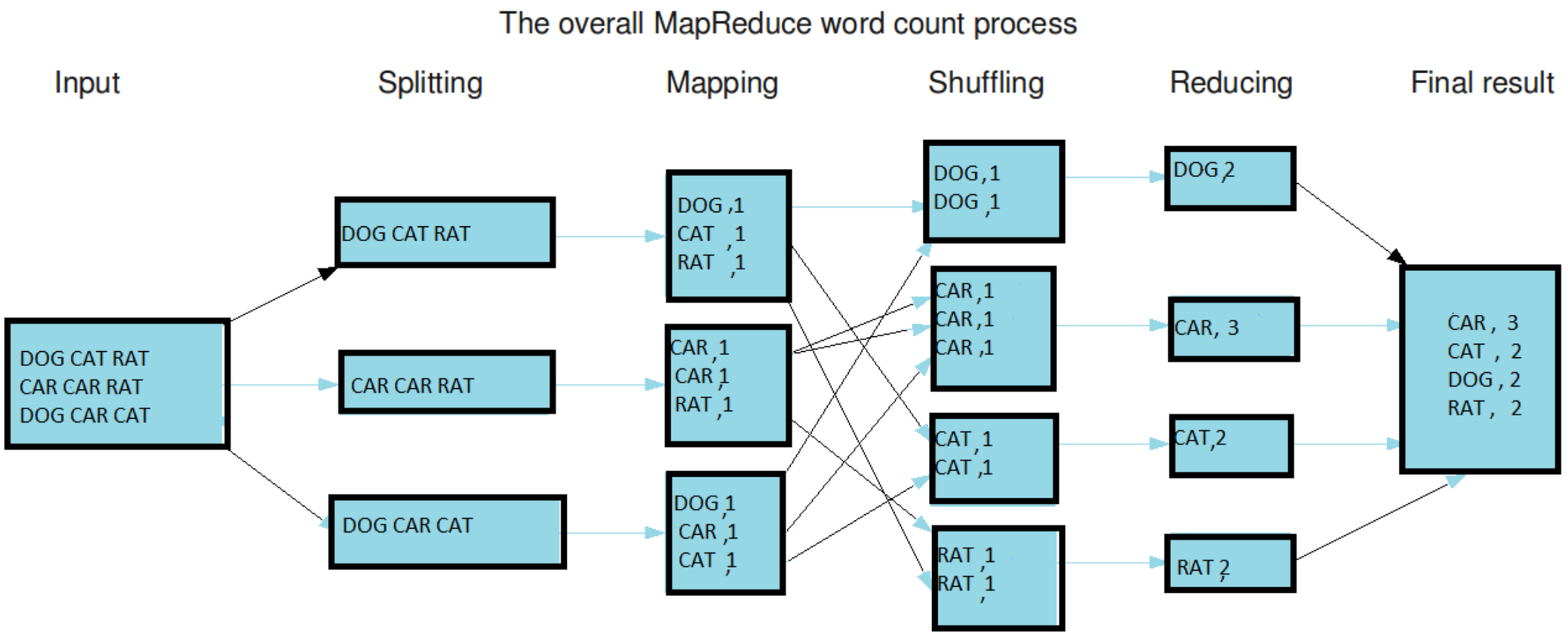
Example
Reducer Input: (Hello,[1,1])
Reducer Output: (Hello,2)

6. Output Phase

Final reducer output is stored back into HDFS
Output Files - Each reducer generates separate output files.
Example:

part-00000
part-00001
part-00002

6.4 MapReduce WordCount Example



The Word Count program is the most common example used to understand how MapReduce works.
Goal is to Count how many times each word appears in a file

Step 1: Input Splitting

The input file is divided into smaller splits.

Split 1 - DOG CAT RAT

Split 2 - CAR CAR RAT

Split 3 - DOG CAR CAT

Splitting allows:

- Parallel processing
- Multiple mappers to work simultaneously

1 Split → 1 Mapper

Step 2: Mapping Phase

Each mapper processes one split and generates:
Intermediate Key-Value Pairs

Format:(word,1)

Mapper 1 Output

Input: DOG CAT RAT
Output:
(DOG,1)
(CAT,1)
(RAT,1)

Mapper 2 Output

Input:CAR CAR RAT
Output:

(CAR,1)
(CAR,1)
(RAT,1)

Mapper 3 Output
Input:DOG CAR CAT

Output:

(DOG,1)
(CAR,1)
(CAT,1)

Step 3: Shuffle Phase

The shuffle phase transfers mapper outputs to reducers.
All identical keys are grouped together.
Grouped Intermediate Data

DOG → (1,1)
CAR → (1,1,1)
CAT → (1,1)
RAT → (1,1)

Reducers must receive All values belonging to the same key
Example - All DOG values must go to the same reducer.

Step 4: Sorting Phase

Data is sorted and organized by keys.
Sorted Data

CAR → [1,1,1]
CAT → [1,1]
DOG → [1,1]
RAT → [1,1]

Purpose of Sorting is to Make aggregation easier for reducers.

Step 5: Reducing Phase

Reducers aggregate grouped values.

Reducer Calculations
DOG - 1 + 1 = 2
CAR - 1 + 1 + 1 = 3
CAT - 1 + 1 = 2
RAT - 1 + 1 = 2

Reducer Output
(DOG,2)
(CAR,3)
(CAT,2)
(RAT,2)

Part VII - Hadoop Use Cases

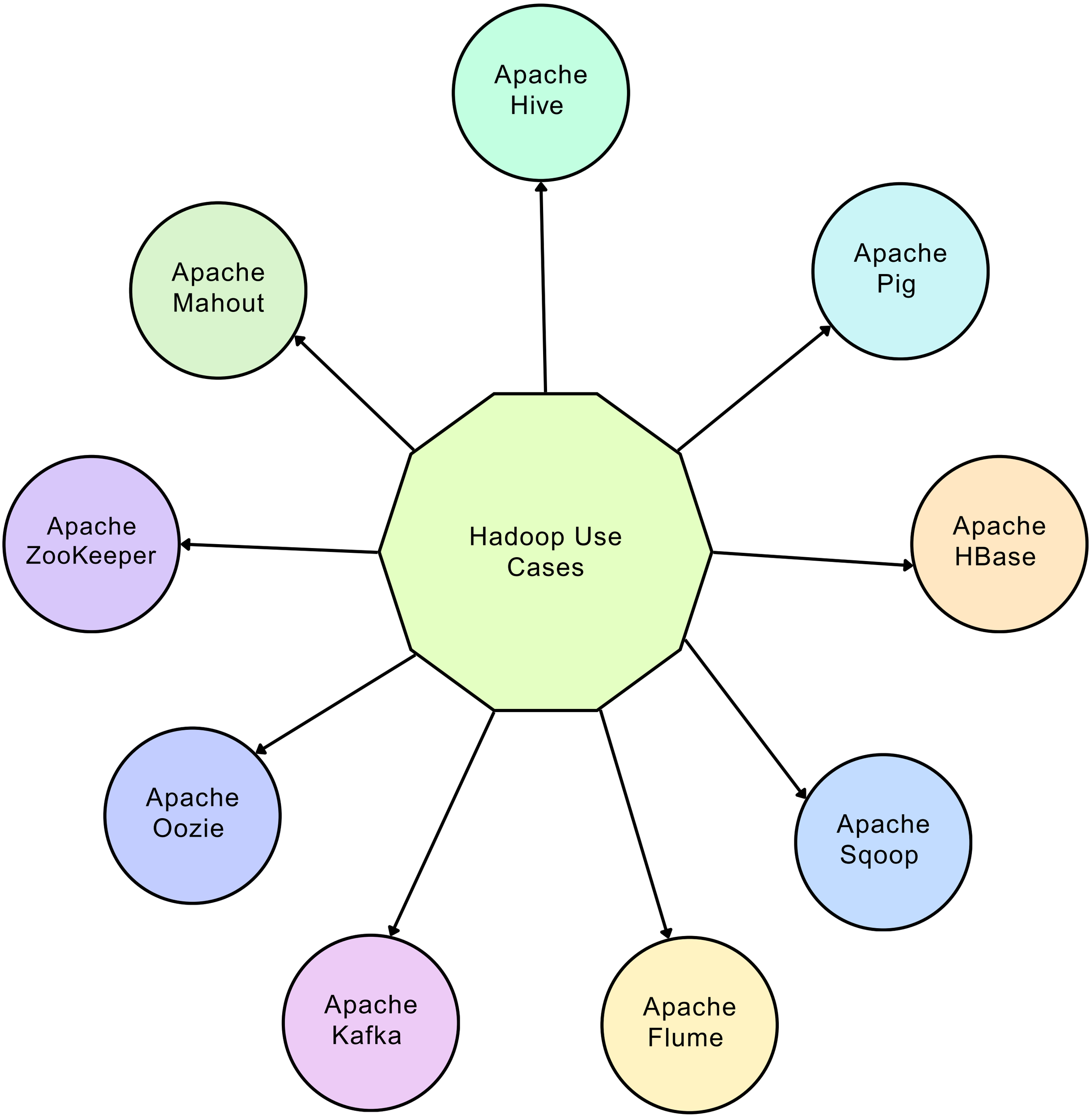
1. Banking and Financial Services	Fraud Detection Analyzing transaction patterns to detect suspicious activities.
	Risk Analysis Processing market and customer data for financial risk prediction.
	Credit Scoring Analyzing customer history and spending behavior.
	Algorithmic Trading Processing real-time market datasets for trading strategies.
2. Retail and E-Commerce	Recommendation Systems Suggesting products based on Purchase history, Browsing behavior and User preferences
	Customer Analytics Understanding customer behavior patterns
	Inventory Management Predicting product demand and stock optimization.
	Dynamic Pricing Adjusting prices using demand and market trends.

3. Healthcare and Life Sciences	Patient Data Analysis Analyzing electronic health records (EHRs).
	Disease Prediction Predictive analytics for disease diagnosis.
	Genomic Data Processing Handling massive DNA sequencing datasets.
	Medical Research Large-scale clinical data analytics.
4. Telecommunications	Call Detail Record (CDR) Analysis Analyzing customer calling patterns.
	Network Optimization Monitoring network traffic and performance.
	Customer Churn Prediction Identifying customers likely to leave services.
	Fraud Detection Detecting unauthorized usage and telecom fraud.
5. Social Media Analytics	Sentiment Analysis Analyzing opinions from posts and comments.
	Trend Detection Identifying viral topics and hashtags.
	User Behavior Analysis Studying engagement patterns.
	Advertisement Targeting Personalized advertisement recommendations.

6. Log and Clickstream Analytics	Website Traffic Analysis Tracking user activity on websites.
	Clickstream Analysis Analyzing navigation paths and user journeys.
	Error Log Processing Detecting application failures and anomalies.
	Security Monitoring Identifying suspicious activity patterns.
7. Cybersecurity	Threat Detection Identifying malicious activitie
	Intrusion Detection Monitoring suspicious network behavior.
	Security Log Analytics Analyzing firewall and server logs.
	Malware Detection Large-scale behavioral analysis.
8. Media and Entertainment	Content Recommendation Movie/music recommendation systems.
	Video Analytics Processing streaming and viewing data.
	Audience Analysis Understanding user preferences and trends.
	Advertisement Analytics Targeted advertising optimization.

9. Internet of Things (IoT)	Sensor Data Processing Analyzing real-time sensor streams.
	Predictive Maintenance Detecting equipment failures before breakdowns.
	Smart City Analytics Traffic, pollution, and utility monitoring.
	Industrial Monitoring Machine and production analytics.
10. Government and Public Sector	Census Data Processing Analyzing population data.
	Smart Governance Data-driven policy decisions.
	Crime Analytics Pattern detection in criminal activities.
	Disaster Management Analyzing weather and emergency data.
11. Machine Learning and AI Pipelines	Data Preprocessing Cleaning and transforming massive datasets.
	Feature Engineering Generating ML features from Big Data.
	Training Data Storage Managing large AI datasets.
	Distributed ML Workflows Integration with Spark and ML frameworks.

Part VIII - Hadoop Ecosystem



8.1 Apache Hive

Apache Hive is a data warehouse infrastructure built on Hadoop that provides SQL-like querying capabilities.
It uses Query Language called HiveQL(HQL)

It is Used for:
- Data analysis
- Batch querying
- Reporting
- ETL operations

Features
- SQL-like interface
- Handles large datasets
- Supports partitioning
- Supports batch analytics

Use Cases
- Data warehousing
- Business intelligence
- Batch analytics

Hive Components
- Hive Driver
- Compiler
- Metastore
- Execution Engine

Advantages
- Easy for SQL users
- Scalable querying
- Integrates with Hadoop ecosystem

Limitations
- High latency
- Not suitable for real-time queries

8.2 Apache Pig

Apache Pig is a high-level data flow scripting platform for processing large datasets.
It Uses Language Pig Latin

Purpose
- Scripting
- Simplifies MapReduce Programming

Features
- Easy scripting
- Handles structured and semi-structured data
- Automatic optimization

Common Operations
- LOAD
- FILTER
- GROUP
- JOIN
- FOREACH

Use Cases
- ETL processing
- Log analysis
- Data transformation

Advantages
- Less coding than Java MapReduce
- Faster development

Limitations
- Not suitable for real-time analytics

8.3 Apache HBase

Apache HBase is a distributed NoSQL column-family database built on top of HDFS.
It Has Column Oriented Storage

Purpose
- Real-time read/write access
- Random access to Big Data

Main Components
- HMaster
- RegionServer
- ZooKeeper

Features
- Distributed database
- Horizontal scalability
- High availability
- Strong consistency

Features
- Distributed database
- Horizontal scalability
- High availability
- Strong consistency

Advantages
- Fast random access
- Handles sparse data

Limitations
- Complex configuration
- Not ideal for complex joins

8.4 Apache Sqoop

Apache Sqoop is a tool used to transfer data between relational databases and Hadoop.

Purpose
Imports and exports data between:
- MySQL
- Oracle
- PostgreSQL
- HDFS
- Hive
- HBase

Features
- Parallel data transfer
- Supports incremental imports
- Integrates with Hive and HBase

Use Cases
- ETL pipelines
- Data migration
- Data warehousing

Advantages
- Fast bulk transfer
- Database integration

8.5 Apache Flume

Apache Flume is a distributed service for collecting and transferring large amounts of log data into Hadoop.

Purpose
Used for:
- Log ingestion
- Streaming log collection

Flume Architecture
Source - Receives data.
Channel - Temporary storage.
Sink - Transfers data to destination.

Features
- Reliable log collection
- Scalable ingestion
- Fault-tolerant design

Use Cases
- Server log collection
- Event data ingestion
- Streaming analytics pipelines

Advantages
- Distributed ingestion
- Reliable delivery

8.6 Apache Kafka

Apache Kafka is a distributed event streaming platform used for real-time data pipelines and streaming applications.

Purpose
- Real-time streaming
- Event processing
- High-throughput messaging

Main Components
Producer - Sends messages.
Broker - Stores messages.
Consumer - Reads messages.

Features
- High throughput
- Fault tolerance
- Distributed messaging
- Real-time streaming

Use Cases
- Real-time analytics
- Log streaming
- IoT pipelines
- Event-driven systems

Advantages
- Extremely scalable
- Low latency

8.7 Apache Oozie

Apache Oozie is a workflow scheduler system for managing Hadoop jobs.

Purpose
Schedules and manages:

- MapReduce jobs
- Hive jobs
- Pig jobs
- Sqoop jobs

Workflow Types
Workflow Jobs - Sequential execution.
Coordinator Jobs - Time/data-triggered workflows.

Features

- Workflow automation
- Dependency management
- Scheduling support

Use Cases

- ETL scheduling
- Batch workflow automation

Advantages

- Centralized workflow management
- Hadoop ecosystem integration

8.8 Apache ZooKeeper

Apache ZooKeeper is a distributed coordination service for managing distributed applications.

Purpose
- Synchronization
- Configuration management
- Leader election

Features
- Centralized coordination
- Distributed synchronization
- High availability

Use Cases
- Cluster coordination
- Distributed locking
- Service discovery

Used By
- HBase
- Kafka
- Hadoop HA systems

Advantages
- Reliable coordination
- Fault tolerance

8.9 Apache Mahout

Apache Mahout is a scalable machine learning library for Big Data analytics.

Purpose
Provides distributed ML algorithms.

Supported Algorithms
- Recommendation Systems
- Clustering
- Classification
- Collaborative Filtering

Features
- Distributed ML processing
- Scalable algorithms
- Hadoop integration

Use Cases
- Recommendation engines
- Customer segmentation
- Predictive analytics

Advantages
- Big Data ML support
- Parallel computation

THANK YOU